The Journal of Open Source Software

HAdaptiveIntegration.jl: Adaptive numerical integration over simplices and orthotopes

Luiz Faria [1] **and Zoïs Moitier** [2¶]

1 POEMS, CNRS, Inria, ENSTA, Institut Polytechnique de Paris, 91120 Palaiseau, France **2** Inria, Unité de Mathématiques Appliquées, ENSTA, Institut Polytechnique de Paris, 91120 Palaiseau, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Gabriele Bozzola

Reviewers:

- @ArrogantGao
- @jipolanco

Submitted: 27 May 2026
Published: unpublished

Summary

`HAdaptiveIntegration.jl` is a `Julia` (Bezanson et al., 2017) package for automatic adaptive numerical integration on multidimensional simplices and axis-aligned orthotopes. It approximates integrals of the form

$$I = \int_\Omega f(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$$

where $f\colon \mathbb{R}^d \to \mathbb{T}$ takes values in a normed real vector space (*e.g.* $\mathbb{T} = \mathbb{R}$, $\mathbb{C}$, $\mathbb{R}^n$), and $\Omega \subset \mathbb{R}^d$ is a simplex (triangles, tetrahedra, etc.) or an axis-aligned orthotope (rectangle, cuboid, etc.). `HAdaptiveIntegration` combines uniform subdivision with embedded cubature to return both an integral estimate and an a posteriori error estimate.

Its main features are:

- Adaptive integration over simplices and orthotopes of arbitrary dimension;
- Efficient tabulated cubature rules for low-dimensional domains;
- Support for user-defined cubature rules and subdivision strategies;
- Arbitrary-precision arithmetic.

Usage

The package is centered on a single function, `integrate(f, domain; kwargs...)`, together with constructors for common domains. The workflow has two steps:

Create a domain using `Simplex` (defined by its vertices) or `Orthotope` (defined by its lower and upper corners), with aliases `Segment`, `Triangle`, `Tetrahedron`, `Rectangle`, `Cuboid`.

```
using HAdaptiveIntegration
segment     = Segment(0, 1)
triangle    = Triangle((0,0), (1,0), (0,1))
rectangle   = Rectangle((0,0), (1,1))
simplex4    = Simplex((0,0,0,0), (1,0,0,0), (0,1,0,0), (0,0,1,0), (0,0,0,1))
```

Integrate using the same interface for both domain types:

```
f(x) = cis(sum(x)) / (sum(abs2, x) + 1e-2)
I, E = integrate(f, triangle)
I, E = integrate(f, rectangle)
```

The function returns a pair `(I, E)`, where `I` is the integral estimate and `E` is its a posteriori error estimate. The main stopping-condition keywords are `atol`, `rtol`, and `maxsubdiv`.

The Journal of Open Source Software

Statement of need

Adaptive numerical integration is a fundamental building block in scientific computing, including finite-element and boundary-element methods and parameter studies over reference or physical cells. To our knowledge, no `Julia` package supported adaptive integration on simplices before this work. `HAdaptiveIntegration` fills this gap while also providing specialized tabulated rules for higher-order accuracy, a single unified interface for both simplices and orthotopes, user-extensible cubature, and optional arbitrary-precision arithmetic.

State of the field

`HAdaptiveIntegration` is complementary to the existing `Julia` ecosystem. `QuadGK.jl` (Johnson, 2013) remains the right choice in one dimension. `HCubature.jl` (Johnson, 2017) is often preferable for medium- to high-dimensional orthotopes, where its estimator-driven axis-aligned bisection can outperform uniform subdivision when the integrand varies primarily along one direction. `Cuba.jl` (Hahn, 2005) is better suited for high-dimensional problems where stochastic methods dominate. The distinct contribution of `HAdaptiveIntegration` is support for simplices of arbitrary dimension together with orthotopes under one API, with efficient tabulated rules in low dimensions. This combination is not provided by the packages above.

Software design

`HAdaptiveIntegration` has two submodules: `Domain`, which defines integration domains and their subdivision; and `Rule`, which defines integration rules (either computed from explicit formulas or tabulated in decimal format). Its single entry point is `integrate`, whose adaptive algorithm combines embedded cubature with a subdivision strategy.

Embedded cubature

At the core of an adaptive numerical integration method is a cubature pair $(\mathcal{H}, \mathcal{L})$, defined on the reference domain $\hat{\omega}$ (equal to $\{\boldsymbol{x} \in \mathbb{R}^d_+ \mid x_1 + \cdots + x_d \leq 1\}$ for simplices or $[0,1]^d$ for orthotopes) by

$$\mathcal{H}(f) = \sum_{1 \leq i \leq \mathsf{H}} h_i \, f(\boldsymbol{x}_i) \quad \text{and} \quad \mathcal{L}(f) = \sum_{1 \leq i \leq \mathsf{L}} \ell_i \, f(\boldsymbol{x}_i), \qquad \forall f \in \mathscr{C}^0(\hat{\omega}).$$

where $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_{\mathsf{H}} \in \hat{\omega}$ are the cubature points, h_i the $\mathcal{H}$ weights, ℓ_i the $\mathcal{L}$ weights, and $\mathsf{H} > \mathsf{L}$. The $\mathcal{L}$ rule uses a subset of the $\mathcal{H}$ points (hence *embedded*), and the two rules have polynomial exactness orders $k_h > k_\ell$, where the order is the highest degree k for which the rule integrates all polynomials exactly.

For a domain ω with reference map $\phi \colon \hat{\omega} \to \omega$, the *local* estimated integral value I_ω and error E_ω are

$$I_\omega = |\det \mathrm{J}_\phi| \; \mathcal{H}(f \circ \phi) \quad \text{and} \quad E_\omega = |\det \mathrm{J}_\phi| \; \|\mathcal{H}(f \circ \phi) - \mathcal{L}(f \circ \phi)\|,$$

where J_ϕ is the Jacobian of ϕ (which is constant for simplices and axis-aligned orthotopes) and $\|\cdot\|$ is the chosen norm on $\mathbb{T}$.

Each domain type has a default embedded cubature summarized in Table 1.

Table 1: Default embedded cubature rules by domain.

1d. `Segment` (Laurie, 1997)	

2d. `Triangle` (Laurie, 1982)	`Rectangle` (Genz & Malik, 1980)
3d. `Tetrahedron` (Grundmann & Möller, 1978)	`Cuboid` (Berntsen & Espelid, 1988)
nd. `Simplex` (Grundmann & Möller, 1978)	`Orthotope` (Genz & Malik, 1980)

The adaptive algorithm

Given a function f and an initial domain Ω, the adaptive algorithm constructs a sequence of nested partitions. It starts with $\mathcal{P}_0 = \{\Omega\}$ and iterates by

$$\mathcal{P}_{n+1} = [\mathcal{P}_n \setminus \{\omega^*\}] \cup \{\omega_1^*, \dots, \omega_{2^d}^*\}, \qquad \forall n \in \mathbb{N},$$

where ω^* is chosen such that $E_{\omega^*} = \max\{E_\omega : \omega \in \mathcal{P}_n\}$, and $\omega_1^*, \dots, \omega_{2^d}^*$ are subdomains given by a subdivision of ω^*. In dimension d, orthotopes are bisected along each axis and simplices by midpoint edge refinement (Gonçalves et al., 2006), both producing 2^d subdomains.

For the sequence $(\mathcal{P}_n)_{n\in\mathbb{N}}$, we define the global integral value I_n and error E_n estimators by

$$I_n = \sum_{\omega\in\mathcal{P}_n} I_\omega \quad \text{and} \quad E_n = \sum_{\omega\in\mathcal{P}_n} E_\omega.$$

For this type of algorithm, the stopping condition is controlled by three parameters: the absolute tolerance $\texttt{atol} \geq 0$, the relative tolerance $\texttt{rtol} \geq 0$, and the maximum number of subdivisions $n_{\max} \in \mathbb{N}$. The subdivision process stops when

$$E_n \leq \texttt{atol} \quad \text{or} \quad E_n \leq \texttt{rtol}\,\|I_n\| \quad \text{or} \quad n = n_{\max}.$$

At the end, (I_n, E_n) is returned as the integral value and error estimate.

Implementation

The implementation uses a max binary heap from DataStructures.jl to store $\{(\omega, I_\omega, E_\omega) : \omega \in \mathcal{P}_n\}$, ordered by E_ω, for efficient retrieval of the maximum local error. When computing multiple integrals of the same type, the heap can be pre-allocated and passed via the `buffer` keyword to reduce allocations.

Extended precision

As noted above, `integrate` supports arbitrary precision. Only the rules from (Genz & Malik, 1980; Grundmann & Möller, 1978) are generated from explicit formulas; the others are tabulated at quadruple precision and are therefore incompatible with arbitrary precision. `HAdaptiveIntegration` addresses this with the optional `IncreasePrecisionExt` extension, which refines a tabulated rule by solving the polynomial exactness conditions with Newton iterations and automatic differentiation in `ForwardDiff.jl` (Revels et al., 2016).

More precisely, let $\hat{\omega}$ be a reference domain and $(\mathcal{H}, \mathcal{L})$ be an embedded cubature on $\hat{\omega}$ with orders $k_h > k_\ell$. Let $b_1, \dots, b_{K_h}$ be a basis of $\mathbb{P}_{k_h}$ (polynomials of total degree $\leq k_h$) such that $b_1, \dots, b_{K_\ell}$ is a basis of $\mathbb{P}_{k_\ell}$. Define $\boldsymbol{u}^{\mathcal{H},\mathcal{L}} = (\boldsymbol{x}_1, \dots, \boldsymbol{x}_{\mathsf{H}}, h_1, \dots, h_{\mathsf{H}}, \ell_1, \dots, \ell_{\mathsf{L}})$ as the embedded cubature data, and the function $F\colon \mathbb{R}^{(d+1)\mathsf{H}+\mathsf{L}} \to \mathbb{R}^{K_h+K_\ell}$ by

$$F(\boldsymbol{u}^{\mathcal{H},\mathcal{L}}) = \begin{pmatrix} \mathcal{H}(b_1) - \int_{\hat{\omega}} b_1(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \\ \vdots \\ \mathcal{H}(b_{K_h}) - \int_{\hat{\omega}} b_{K_h}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \\ \mathcal{L}(b_1) - \int_{\hat{\omega}} b_1(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \\ \vdots \\ \mathcal{L}(b_{K_\ell}) - \int_{\hat{\omega}} b_{K_\ell}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \end{pmatrix}.$$

Starting from a tabulated rule with $\|F(\boldsymbol{u}^{\mathcal{H},\mathcal{L}})\|_2 = \varepsilon \ll 1$, we seek $\tilde{\boldsymbol{u}}$ with $\|F(\tilde{\boldsymbol{u}})\|_2 = \eta < \varepsilon$ via a least-squares Newton method (Xiao & Gimbutas, 2010, sec. 2.3). Setting $\boldsymbol{u}_0 = \boldsymbol{u}^{\mathcal{H},\mathcal{L}}$, the iteration is

$$\boldsymbol{u}_{p+1} = \boldsymbol{u}_p - \mathrm{J}_F(\boldsymbol{u}_p)^\dagger F(\boldsymbol{u}_p), \tag{N}$$

$\mathrm{J}_F(\boldsymbol{u}_p)$ is the Jacobian matrix of F at the point $\boldsymbol{u}_p$, and $\mathrm{J}_F(\boldsymbol{u}_p)^\dagger$ is the pseudo-inverse. In `Julia`, this is implemented with the `\` operator. The iteration stops when $\|\boldsymbol{u}_{p+1} - \boldsymbol{u}_p\|_2 \le$ `x_atol` (absolute tolerance of successive iterates) or $\|F(\boldsymbol{u}_p)\|_2 \le$ `f_atol` (absolute tolerance of function value) or $p = p_{\max}$ (maximum number of iterations).

Remark. The monomial basis is convenient but poorly conditioned, so achieving precision ε requires `BigFloat` arithmetic at a higher internal precision $\eta < \varepsilon$; an L^2-orthogonal basis would improve conditioning.

Remark. Symmetry is not enforced in the Newton iteration; a rule refined from precision ε to $\eta < \varepsilon$ therefore retains its symmetry only up to ε.

Research impact statement

The repository includes API documentation, examples (buffer pre-allocation, `callback` mechanism, custom cubature, arbitrary-precision workflows), an automated test suite, and continuous integration covering tests, documentation, and linting. `HAdaptiveIntegration` has been featured in the Julia world newsletter, interfaced via Integrals.jl (SciML ecosystem), used as a backend in (Anderson et al., 2024), and adopted as a dependency of `Meshes.jl`.

Example gallery

We showcase the package on integrands with localized features constructed from the mollified integrand

$$\delta_\epsilon(\phi(\boldsymbol{x})) = \frac{1}{\varepsilon^c}\eta\left(\frac{\phi(\boldsymbol{x})}{\varepsilon}\right), \qquad \eta(r) = \frac{1}{\sqrt{2\pi}}e^{-r^2/2},$$

where ϕ is a level-set function vanishing on the feature $\Gamma = \phi^{-1}(0)$ and c is the codimension of Γ (so the total mass stays $\mathcal{O}(1)$ as $\varepsilon \to 0$). We consider three canonical geometries:

- **Point:** $\phi(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{x}_0\|_2$, codimension $c = d$;
- **Hypersphere:** $\phi(\boldsymbol{x}) = \|\boldsymbol{x}\|_2^2 - r^2$, codimension $c = 1$;
- **Hyperplane:** $\phi(\boldsymbol{x}) = x_1 - x_*$, codimension $c = 1$.

We set $\boldsymbol{x}_0 = \frac{1}{\pi}(1, \ldots, 1)$, $r = 1/2$, and $x_* = 1/\pi$ so that the features are well contained in the domain. The hyperplane is axis-aligned, making it a useful benchmark for comparing uniform versus estimator-driven subdivision. Convergence plots sweep $\texttt{rtol} = 10^{-i}$ ($i = 1, \ldots, 10$; up to 8 in 3D), recording the total number of evaluations N, the returned error estimate, and the actual error against a reference solution computed at $\texttt{rtol} = 10^{-12}$.

Simplices

Figure 1 shows convergence on the unit triangle ($d = 2$, Radon-Laurie rule (Laurie, 1982)) and tetrahedron ($d = 3$, Grundmann-Möller rule (Grundmann & Möller, 1978), available in arbitrary dimension). Two observations hold across all features and both geometries: the estimated error reliably tracks the actual error, confirming a sound a posteriori indicator; and once the feature is resolved, errors follow $\mathcal{O}(N^{-(k+1)/d})$ with $k = k_h$ or k_ℓ and the exponent implied by $N \propto h^{-d}$.

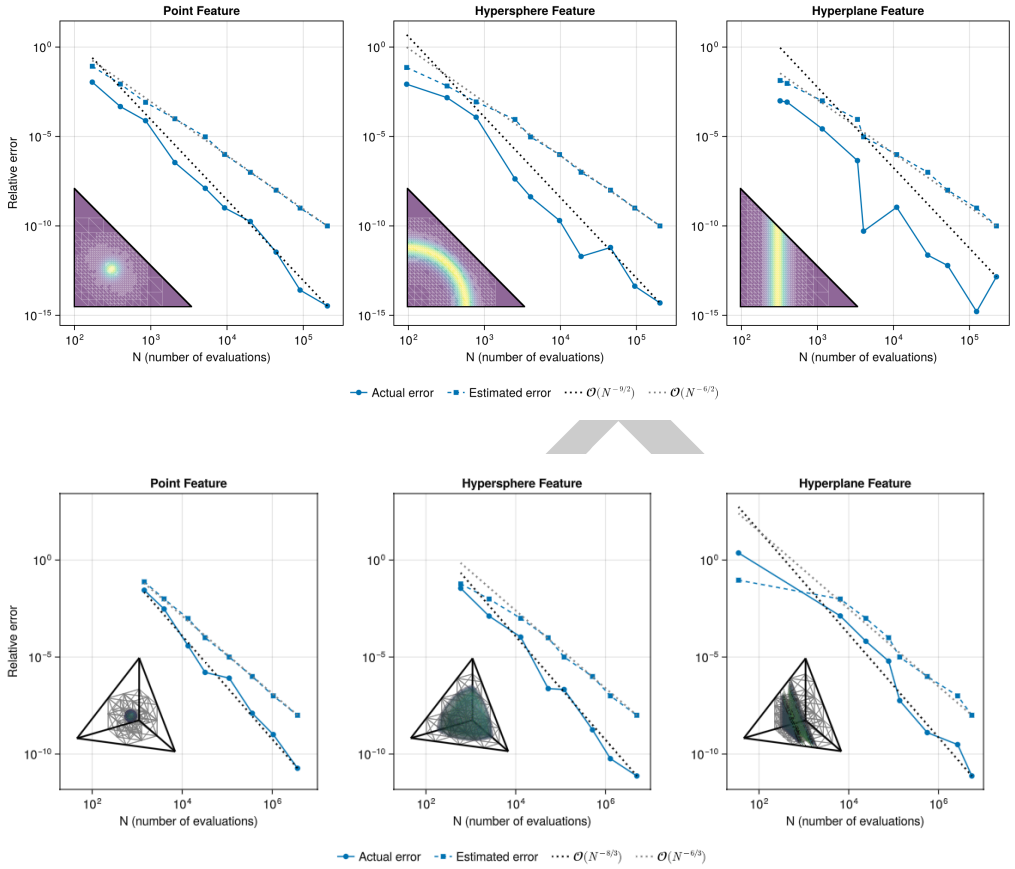

Figure 1: Convergence of the actual and estimated errors for the point, hypersphere, and hyperplane features on the unit triangle (top) and unit tetrahedron (bottom) versus the number of evaluations (N). Insets show representative adaptive sub-domains and integrand visualizations.

Orthotopes and comparison with `HCubature.jl`

Figure 2 compares `HAdaptiveIntegration` against `HCubature.jl` on the unit square and cube. In 2D, both use the Genz-Malik rule (Genz & Malik, 1980), isolating the subdivision strategy; in 3D, the cubature rules also differ (Berntsen-Espelid (Berntsen & Espelid, 1988) for `HAdaptiveIntegration`). For the point and hypersphere features the solvers are comparable in both dimensions. For the hyperplane, `HCubature.jl` has a clear advantage because its estimator refines exclusively along x_1 rather than bisecting uniformly in all d directions, and this gap grows with d, motivating anisotropic splitting as future work.

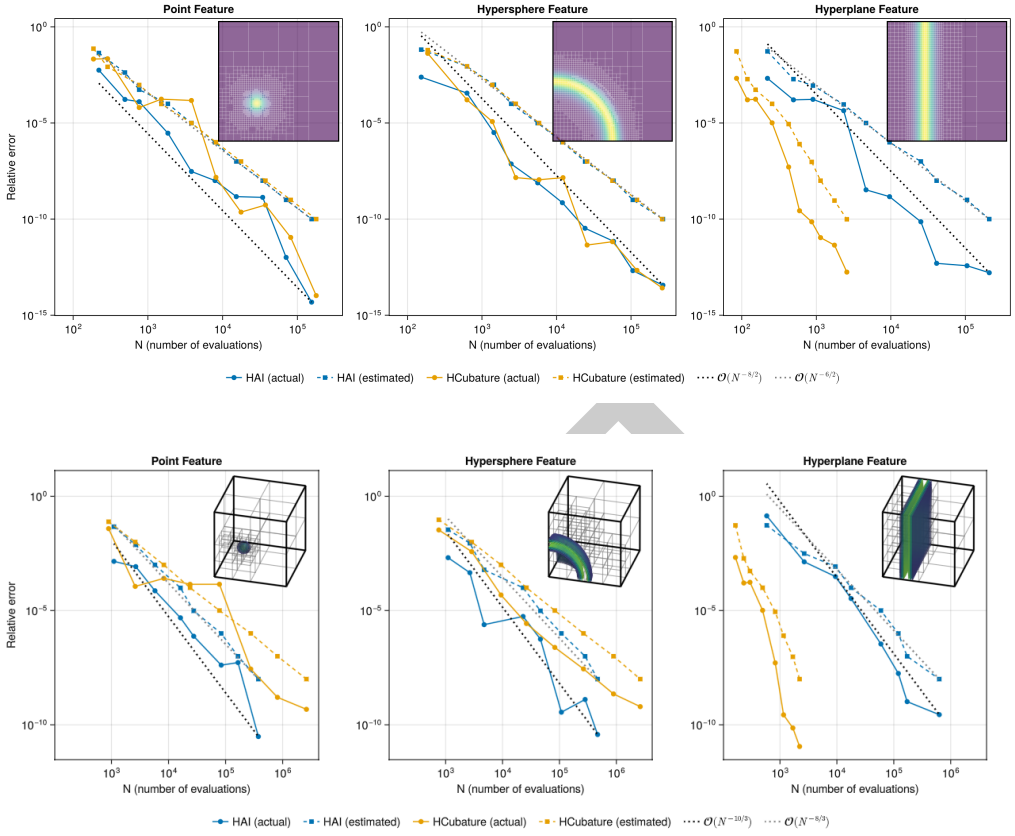

Figure 2: Convergence of the actual and estimated errors for the point, hypersphere, and hyperplane features on the unit square (top) and the unit cube (bottom), comparing `HAdaptiveIntegration` (HAI) with `HCubature.jl`. Insets show representative adaptive sub-domains and integrand visualizations.

AI usage disclosure

Generative AI was used to help develop the code and to draft and edit parts of this manuscript. All text produced with AI assistance was reviewed, revised, and verified by the authors before inclusion in the code or paper.

References

Anderson, T. G., Faria, L. M., & Pérez-Arancibia, C. (2024). Inti.jl. In *GitHub repository*. GitHub. https://github.com/IntegralEquations/Inti.jl

Berntsen, J., & Espelid, T. O. (1988). On the construction of higher degree three-dimensional embedded integration rules. *SIAM J. Numer. Anal.*, *25*(1), 222–234. https://doi.org/10.1137/0725016

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Rev.*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Genz, A. C., & Malik, A. A. (1980). Remarks on algorithm 006: An adaptive algorithm for numerical integration over an N-dimensional rectangular region. *J. Comput. Appl. Math.*, *6*, 295–302. https://doi.org/10.1016/0771-050X(80)90039-X

Gonçalves, E. N., Palhares, R. M., Takahashi, R. H. C., & Mesquita, R. C. (2006). Algorithm 860: SimpleS – an extension of Freudenthal's simplex subdivision. *ACM Trans. Math.*

Softw., *32*(4), 609–621. https://doi.org/10.1145/1186785.1186792

Grundmann, A., & Möller, H. M. (1978). Invariant integration formulas for the n-simplex by combinatorial methods. *SIAM J. Numer. Anal.*, *15*, 282–290. https://doi.org/10.1137/0715019

Hahn, T. (2005). Cuba—a library for multidimensional numerical integration. *Computer Physics Communications*, *168*(2), 78–95. https://doi.org/10.1016/j.cpc.2005.01.010

Johnson, S. G. (2013). QuadGK.jl: Gauss–Kronrod integration in Julia. In *GitHub repository*. GitHub. https://github.com/JuliaMath/QuadGK.jl

Johnson, S. G. (2017). The HCubature.jl package for multi-dimensional adaptive integration in Julia. In *GitHub repository*. GitHub. https://github.com/JuliaMath/HCubature.jl

Laurie, D. P. (1982). CUBTRI: Automatic cubature over a triangle. *ACM Trans. Math. Softw.*, *8*, 210–218. https://doi.org/10.1145/355993.356001

Laurie, D. P. (1997). Calculation of Gauss-Kronrod quadrature rules. *Math. Comput.*, *66*(219), 1133–1145. https://doi.org/10.1090/S0025-5718-97-00861-2

Revels, J., Lubin, M., & Papamarkou, T. (2016). Forward-mode automatic differentiation in Julia. *arXiv:1607.07892 [Cs.MS]*. https://arxiv.org/abs/1607.07892

Xiao, H., & Gimbutas, Z. (2010). A numerical algorithm for the construction of efficient quadrature rules in two and higher dimensions. *Comput. Math. Appl.*, *59*(2), 663–676. https://doi.org/10.1016/j.camwa.2009.10.027